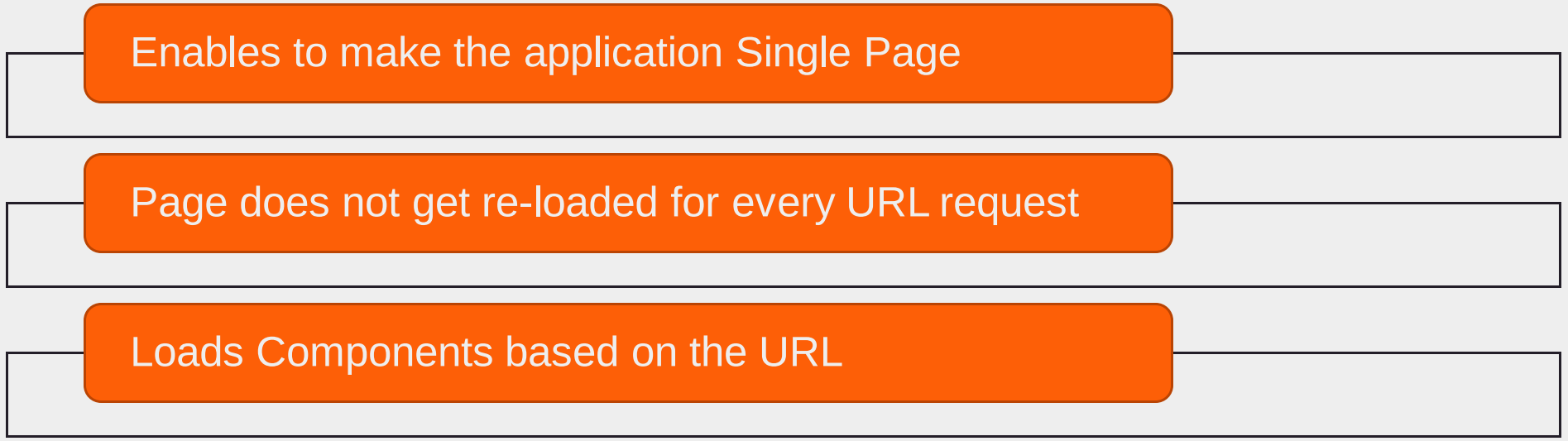
Routing in Angular

Persistent Interactive | Persistent University

Key Learning Points

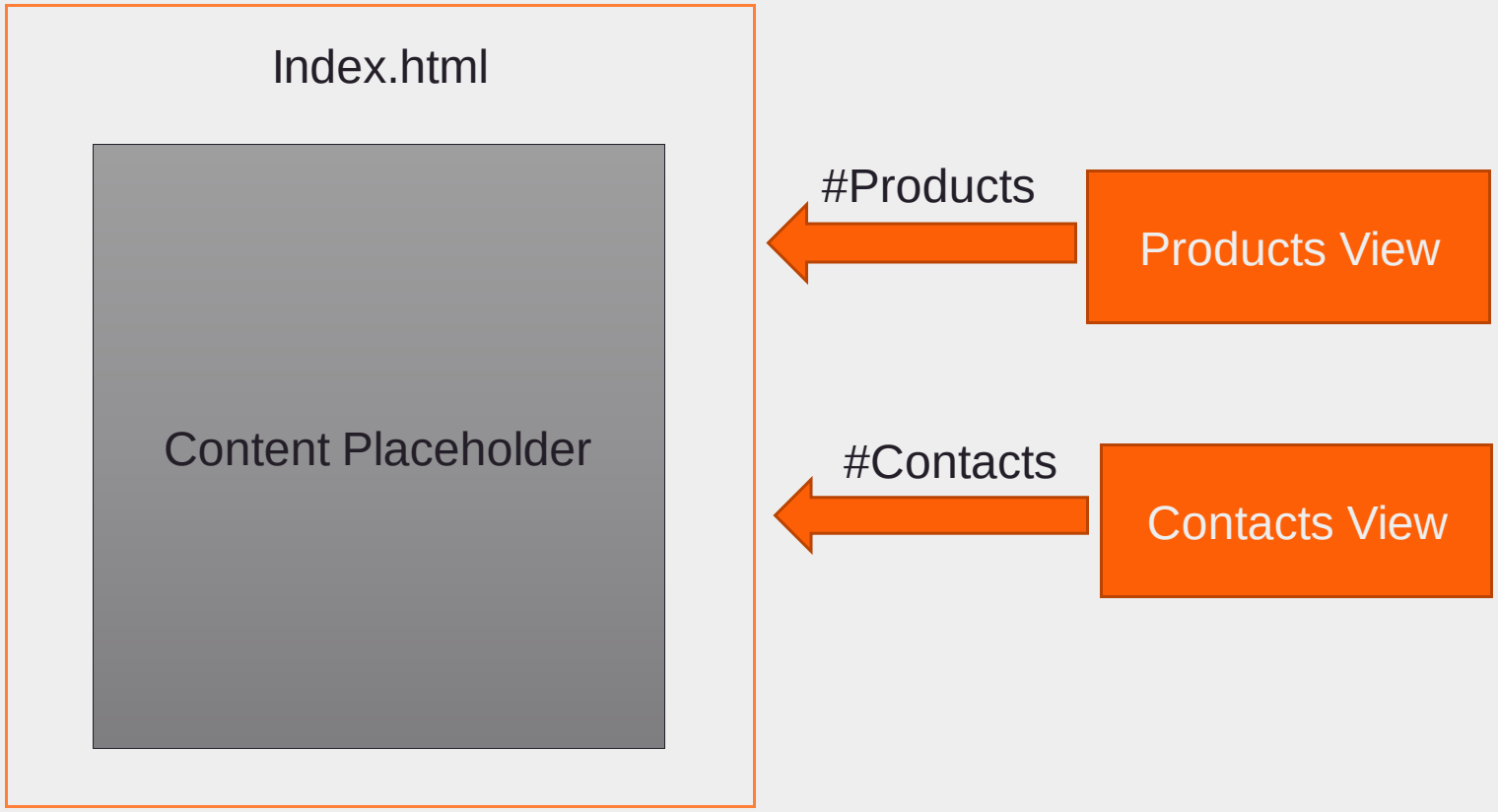
- What is Client-side Routing?
- Routing Configurations
- Defining Child Routes
- Configure Default Route
- Handling Invalid routes
- Route Parameters

Features of Routing



Introduction to Routing

- Loading content dynamically based on the links which user clicks on.



Persistent

Getting started

- Use the **@angular\router** module

- Specify the mapping of URL with Components

Important imports

- In the **module.ts**

```
import {RouterModule,Routes} from '@angular/router';
```

- Add to the imports array.

```
@NgModule({
  imports:      [ BrowserModule,FormsModule,RouterModule.forRoot(routes),HttpModule ],
  declarations: [ AppComponent,UserFormComponent,MenuComponent,ProductsComponent,Welcom
  bootstrap:    [ AppComponent ],
  providers:[CartService]
})
```

Why RouterModule.forRoot()?

- When we need a single instance of a service from the module to be used in the Root module as well, we use the method **forRoot()**
- There should only be **one RouterService** for a single Angular application.
- **forRoot()** will initialize that service and register it to DI together with some route config.
- Since Root Module (AppModule) is the entry point of an application, the Routerservice is initialized here.
- The forRoot() function should only be used for creating the root modules' routes.

Route interface

- Create an array of routes with the mapping of URL and component

```
const routes:Routes=[
  {
    path:'products',
    component:ProductsComponent
  },
  {
    path:'userform',
    component:SignUpFormComponent
  }
]
```

```
interface Route {
  path?: string;
  component?: Type|string;
  ...
}
```

Default route

- A path of “”(empty string) is used to provide a default route for the application.

```
{
  path:'',
  component:LoginComponent
},
```

- We can also use redirect property to point to another path.

```
const routes: Routes = [
  {
    path:'',
    redirectTo:'/login',
    pathMatch:'full'
  },
```

This property needs to be set while redirecting the default route. This ensures exact matching of empty string and not partial one. For other redirects this can be skipped.

RouterLink

- **RouterLink** takes care of generating an href attribute for us that the browser needs to make linking to other sites work.

```
<a RouterLinkActive="active" routerLink="static">Static Link</a>
```

- Supports property binding with [routerLink]

```
<ul class="nav nav-pills">
  <li *ngFor="let item of menuItems" role="presentation"
      <a [routerLink]='item | lowercase' >{{item}}</a></li>

</ul>
```

Router Outlet

- The **app.component** template will have the <router-outlet></router-outlet>

```
@Component({
  selector: 'my-app',
  template: `<h1>Hello {{name}}</h1>
              <menu></menu>
              <router-outlet></router-outlet>`,
})
```

** Wildcard

- The ** path in the last route is a wildcard.
- The router will select this route if the requested URL doesn't match any paths for routes defined earlier in the configuration.
- This is useful for displaying a "404 - Not Found" page or redirecting to another route.

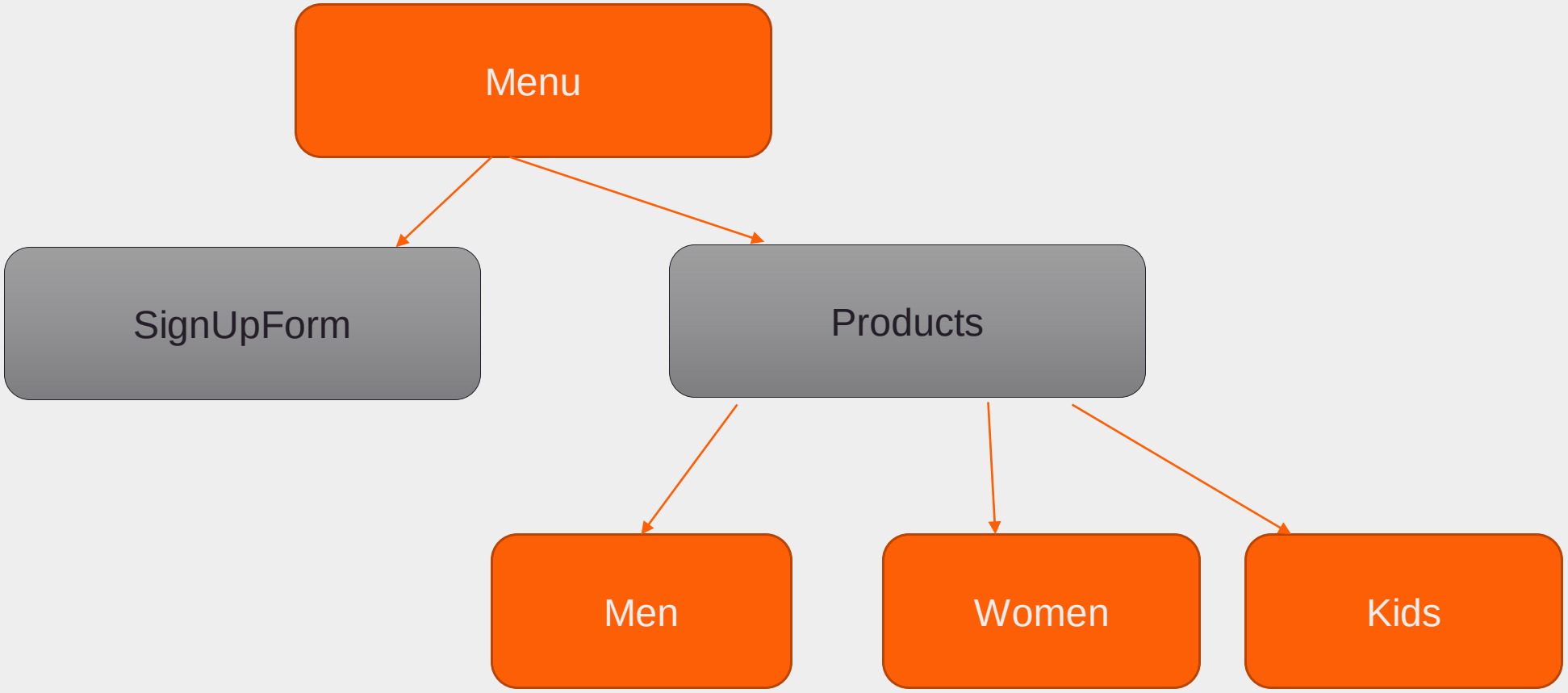
Ordering of routes matters

- **The order of the routes in the configuration matters** and this is by design.

- The router uses a **first-match wins** strategy when matching routes, so more specific routes should be placed above less specific routes.

```
{
    path: "**",
    component: PageNotFoundComponent
}
```

Child Routes



Persistent

Child Routes implementation

- Use the property ‘children’ to specify the child routes.

```
const routes:Routes=[
  {
    path:'products',
    component:ProductsComponent,
     children: [
      { path: 'men', component: MenProductsComponent },
      { path: 'women', component: WomenProductsComponent }
    ]
  },
```

- The **routerLink** in the sub menu.

```
<li role="presentation" class="active"><a [routerLink]="['/products/men']">Men</a></li>
<li role="presentation" class="active"><a [routerLink]="['/products/women']">Women</a></li>
```

Passing Route Parameters

- Second parameter in the **RouterLink** can b used for route parameters.

```
<a [routerLink]="['/products/women',2]">Women</a>
```

- If referring to a variable with *ngFor, use below syntax.

```
<a [routerLink]="[item |lowercase,2]">{{item}}</a>
```

Change the routes

- Specify the parameter in the route mapping.

```
const routes:Routes=[
  {
    path:'products',
    component:ProductsComponent,
     children: [
      { path: 'men/:id', component: MenProductsComponent },
      { path: 'women/:id', component: WomenProductsComponent }
    ]
  },
```

Access the route parameter in the Component

- Use of module **ActivatedRoute**

```
import { ActivatedRoute } from '@angular/router';
```

- Inject in the constructor

```
constructor(private _activateroute:ActivatedRoute) { }
```

Use of Observables

- Define a Subject

```
export class WomenProductsComponent
    id:number
    paramSubject:any;
```

- Subscribe to the route parameters.

```
ngOnInit() {
  this.paramSubject = this._activateroute.params.subscribe(params=>{
      this.id = parseInt(params['id'],10)
  });
```

Programmatic router navigation

- Inject the Router Service

```
constructor(private _router:Router) { }
```

- How are we getting this service? Is it added to any list of providers?
- Navigate using the router service.
- Navigate([]) or navigateByUrl(“”)

```
signup():void{
    this._router.navigate(['/userform']);
  }
```

Single path route

- We have seen a simple route path so far as shown below.

```
{path: 'search', component: SearchComponent},
```

- The corresponding navigate() will be

```
this.router.navigate(['search']);
```

Link params array

- We can also have a path as shown below.

```
{path: 'search/foo/moo', component: SearchComponent},
```

- The corresponding navigate() will be

```
this.router.navigate(['search', 'foo', 'moo']);
```

- We can also use a variable as:

```
let part = "foo";
this.router.navigate(['search', part, 'moo']);
```

- This is useful when dealing with parametrized routes

Optional parameters

- Suppose a path configured takes a parameter then below is used..

```
{path: 'search/:term', component: SearchComponent}
```

- And the navigation call is as below.

```
this.router.navigate(['search', term]);
```

- But this makes it mandatory to pass the parameter. Better approach is pass an object

Optional parameters

- Define the path configured without a parameter.

```
{path: 'search', component: SearchComponent}
```

- And the navigation call is as below.

```
this.router.navigate(['search', {term: term}]);
```

- This is called Matrix URL notation, a series of optional key=value pairs separated with the semicolon ; character.

Avoid URL change

- It is possible to avoid changing the URL every time user navigates the menu by using the ‘**skiplocationflag**’.

```
<a [routerLink]="[item | lowercase,5]" skipLocationChange>{{item}}</a>
```

Routing strategies

- HashLocationStrategy

```
RouterModule.forRoot(routes,{ useHash: true })
```

- PathLocationStrategy
  - This is the default strategy in Angular.

Query Parameters

- Query parameters allow you to pass optional parameters to a route as below.
- **localhost:3000/product-list?page=2**
- Note that this is different from route params where the format is:
- **localhost:3000/product-list/5**

- Query params are optional, so the route will load even if params are none.
- However, route params are mandatory to load the route.

Passing Query Parameters

- Use the [queryParams] directive along with [routerLink] to pass query parameters.

```
<a [routerLink]="['product-list']" [queryParams]="{ page: 99 }">Go to Page 99</a>
```

- Query parameters can also be passed while navigating programmatically.

```
goToPage(pageNum) {
  this.router.navigate(['/product-list'], { queryParams: { page: pageNum } });
 }
```

- Reading Query parameters
- https://angular-2-training-book.rangle.io/handout/routing/query_params.html

How to save Component state

- When user navigates to a different url , then the current component gets destroyed and gets re-created when he again visited that url.
- To save the state before navigating away, there are multiple options.

1. Use services to fetch data and make changes via a service.
2. Use localstorage/sessionstorage
3. Use deactivate routing guard
4. RouteReuse Strategy

Summary: Session

With this we have come to an end of our session, where we discussed about ….

- Using Router Module
- Routing Configuration
- Route Parameters
- Child Routes
- Routing programmatically.

Appendix

References

Thank You

Reference Links

- https://codecraft.tv/courses/angular/routing/overview/
- https://blog.thoughtram.io/angular/2016/06/14/routing-in-angular-2-revisited.html
- Child Routes and Route Parameters
  - https://coryrylan.com/blog/introduction-to-angular-routing
- Need for Router.forRoot()
  - http://blog.angular-university.io/angular2-ngmodule/
- Programmatic Navigation and 404 handler
  - http://blog.angular-university.io/angular2-router/
- Pets Example
  - https://scotch.io/tutorials/routing-angular-2-single-page-apps-with-the-component-router
- 404 issue when a route is refreshed.
  - http://stackoverflow.com/questions/35284988/angular-2-404-error-occur-when-i-refresh-through-browser

Key Contacts :

Persistent University :

- Archana Ghatigar
  archana_ghatigar@persistent.co.in

Persistent

Persistent

Thank you!

Persistent Interactive | Persistent University